Entregables por cada fase

Gonzalo Martínez
gonzalo.martinez@uc.cl



Evaluación del Curso

- **Nota Final =**
  0.05∗Control + 0.20∗Interrogación + 0.25∗Examen +
  0.05∗E1 + 0.10∗E2 + 0.15∗E3 + 0.20∗E4
  - **En consecuencia Nota Final = 50% Pruebas + 50% Proyecto.**
  - **Inasistencia a alguna evaluación, se reemplaza por la nota Examen.**
  - **No hay eximición del Examen.**
- **Proyecto**
  - E1 = 90% trabajo evaluado + **10% evaluación pares**
  - E2 = 80% trabajo evaluado + **20% evaluación pares**
  - E3 = 70% trabajo evaluado + **30% evaluación pares**
  - E4 = **50% trabajo evaluado** + **50% evaluación pares**
- **Condición de aprobación del curso**:
  - **Promedio Evaluaciones (Control, Int, Ex) >= 4.0 AND**
    E4 Trabajo >= 5.0 AND
    Promedio (Evaluación Pares de todas las entregas) >=5.0
- **Si reprobación, Nota Final = Min(Prom_Evaluaciones;Prom_Proyecto)**
- **Si reprueba por E4 y/o Evaluación Pares; Nota Final = 3.0**

Todos los repositorios deben ser compartidos a los siguientes usuarios en github:

gmartinezramirez

E1 Inception (Design Doc)

E2 Elaboration - Walking Skeleton

Iteraciones de n semanas

1 Visión

2 Define and design

3 Product backlog

4 Planning

5 Team card wall/board

6 Develop

7 Potentially shippable increment

8 Demo

9 Final Showcase

Team

Evaluadores

Priority

User Story
User Story
User Story
Epic
Epic
Epic

TO DO | DOING | DONE

Test and review
Analyse
Design
Develop
Deploy

Inspect and adapt

E3 Development

E4 MVP

Team

Evaluadores

Team

Evaluadores

Modern Software Engineering Practices

¿Cómo se evalúa el proyecto?

- Cada aspecto se evalúa en una escala de {1, 0.75, 0.50, 0.25, 0}.

Puntaje	Significado
1	Todo bien, sin observaciones.
0,75	Todo bien, solo observaciones menores.
0,50	En general bien, pero manifesta observaciones mayores o en su defecto una pequeña parte de lo presentado es incorrecto. Nada grave.
0,25	Observaciones mayores, cumple con lo mínimo.
0	Todo mal o no cumple con este aspecto.

Fechas de entrega y presentación

- Las fechas de presentaciones se sortean de forma aleatoria una vez que todos los alumnos se encuentren inscritos con sus respectivos equipos.

Fase	Objetivo	Modalidad
Entrega 1: Inception (Design Doc)	Establecer que problema van a solucionar y como el software a construir va a ayudar en esto	En la semana que corresponda: L1W1 presencial - Grupos que no alcancen presencial presentarán online.
Entrega 2: Elaboration	Refinar aspectos del diseño del software a construir y tener un software mínimo para comenzar la construcción (Walking Skeleton)	En la semana que corresponda: L1W1 presencial - Grupos que no alcancen presencial presentarán online.
Entrega 3: Development	Mostrar el estado actual del desarrollo y cómo han llevado a cabo su proyecto	En la semana que corresponda: L1W1 presencial - Grupos que no alcancen presencial presentarán online.
Entrega 4: MVP	Entrega del software funcionando	En la semana que corresponda: L1W1 presencial - Grupos que no alcancen presencial presentarán online.

Herramientas a utilizar

- **Cada equipo tiene libertad sobre las tecnologías y herramientas a utilizar.**
- **Para la gestión del proyecto pueden usar: Trello (gratis), Jira Cloud (gratis), u otra plataforma.**
- Los equipos deben mostrar su producto software funcionando en producción en algún proveedor cloud.
  - **IMPORTANTE: No deben gastar dinero ni usar tarjetas de crédito. No hay necesidad para esto. Esto queda a voluntad de cada equipo.**
- Para llegar a producción pueden usar proveedores gratuitos tales como: ***Render***, ***Netlify***, ***Vercel***, ***Cloudflare Pages***, ***Github Pages***, ***Cloud66***
- También pueden ocupar proveedores cloud como: AWS, Google Cloud (GCP), Azure. Pero lo malo de estos proveedores es que hay que ingresar una tarjeta de crédito (***en lo posible no hagan esto, no es requerimiento del curso)***.
- **IMPORTANTE: Si no pueden llegar a producción, no hay problema, pero deben avisar de esto al profesor a la brevedad. Última opción para levantar este asunto es en la Entrega 3.**

Consideraciones Material Semestres Pasados

- Se encuentra disponible material que han producido grupos en semestres pasados a lo largo de cada entrega.
- **No obstante, eso no significa que todo este material sea correcto**. **Por lo que cada equipo debe ser crítico con el material entregado** y con el contexto del problema a resolver, lo que funciona para un equipo puede no funcionar para otro.
- **Cada decisión que ustedes tomen como equipo no puede estar justificada con: “es que el equipo X lo hizo”**
- **A medida que pasan los semestres los entregables cambian por lo que deben guiarse con lo expuesto en esta diapositiva.**

Entregables 1era Entrega Inception

Gonzalo Martínez
gonzalo.martinez@uc.cl



E1 Inception

E2 Elaboration - Walking Skeleton

Iteraciones de n semanas

1 Visión

2 Define and design

3 Product backlog

4 Planning

5 Team card wall/board

Team

Evaluadores

Priority

- User Story
- User Story
- User Story
- Epic
- Epic
- Epic

TO DO | DOING | DONE

9 Final Showcase

8 Demo

7 Potentially shippable increment

6 Develop

E4 MVP

Team

Evaluadores

Team

Evaluadores

E3 Development

Inspect and adapt

Test and review

Analyse

Design

Develop

Deploy

Modern Software Engineering Practices

Consideraciones 1era Entrega

- Esta primera entrega se debe aplicar todo lo aprendido en los cursos anteriores de Ingeniería de Software (prerrequisito de este curso) y cursos anteriores.

Checklist Entrega 1

(Tiempo de presentación total 15 a 20 minutos: Son 10 minutos de presentación + 5 a 10 min preguntas)

1. Identificar la razón y justificación del software/solución a construir
   a. Introducción al problema y contexto
   b. Problema y/o motivación
   c. Valor de negocio
2. Identificar a los interesados y posibles usuarios del software
   a. Resumen de los interesados
   b. Resumen de los usuarios
3. Identificar las necesidades a resolver por el software
   a. Descripción del producto/solución a construir
   b. Resumen de rasgos y beneficios
   c. Suposiciones a tomar en el software a construir (por ejemplo, se asume que solo va a funcionar en Chile)
   d. Modelo de proceso
4. Identificar riesgos que pueden afectar el desarrollo del software y también riesgos que pueden afectar el funcionamiento del producto/negocio

Checklist Entrega 1

(Tiempo de presentación total 15 a 20 minutos: Son 10 minutos de presentación + 5 a 10 min preguntas)

5. Detalle de la solución propuesta
    a. Requisitos funcionales (RF) y beneficios que trae la solución de cara al cliente/usuario
    b. Restricciones
    c. Requisitos no funcionales (RNF)
    d. Tecnologías a ocupar (con la debida justificación)
    e. Primera versión de la arquitectura
    f. Primeras maquetas (mockups) de las vistas
6. Plan de trabajo
    g. Detalle de todas las actividades a implementar y desarrollar, ya sea, actividades que no involucran código si no que análisis, tales como, arquitectura, armar su proceso metodológico, revisión de deuda técnica, entre otros y también considerar los RF a implementar por cada semana/iteración
    h. Ustedes deben pensar en la longitud de las iteraciones (2 semanas, 1 semana, 3 semanas, etc) y que esperan tener completado en cada una de ellas.
7. Glosario y anexo
    i. Glosario del dominio a abordar, es decir, por ejemplo, estoy desarrollando un agendamiento de peluquería de mascotas, pero que es una mascota en este contexto ? que es un agendamiento?, etc. El objetivo de esto es no tener dudas acerca de términos claves del problema/negocio/dominio a implementar.

Checklist Entrega 1

1. Identificar la razón y justificación del software/solución a construir
    a. Introducción al problema y contexto
    b. Problema y/o motivación
    c. Valor de negocio
2. Identificar a los interesados y posibles usuarios del software
    a. Resumen de los interesados
    b. Resumen de los usuarios
3. Identificar las necesidades a resolver por el software
    a. Descripción del producto/solución a construir
    b. Resumen de rasgos y beneficios
    c. Suposiciones a tomar en el software a construir (por ejemplo, se asume que solo va a funcionar en Chile)
4. Identificar riesgos que pueden afectar el desarrollo del software y también riesgos que pueden afectar el funcionamiento del producto/negocio

Checklist Entrega 1

5. Detalle de la solución propuesta
    a. Requisitos funcionales (RF) y beneficios que trae la solución de cara al cliente/usuario
    b. Requisitos no funcionales (RNF)
    c. Tecnologías a ocupar (con la debida justificación)
    d. Primera versión de la arquitectura
6. Plan de trabajo
    e. Detalle de todas las actividades a implementar y desarrollar, ya sea, actividades que no involucran código si no que análisis, tales como, arquitectura, armar su proceso metodológico, revisión de deuda técnica, entre otros y también considerar los RF a implementar por cada semana/iteración

Pauta Entrega 1 - Inception

Introducción y contexto	Problema y/o motivación	Valor de negocio	**Nota Razón y justificación**
1	**2**	**3**	**7,00**
1	1	1	**7,00**

Resumen interesados	Resumen de usuarios	**Nota Interlocutores**
3	**3**	**7,00**

Resumen de rasgos	Suposiciones	Modelo de proceso	**Nota Necesidades**
1	**2**	**3**	**7,00**
1	1	1	**7,00**

Riesgos	RF y beneficios	Restricciones	RF no funcionales	Plan de trabajo	Tecnología + Arquitectura	Nota Requisitos
0,5	**2**	**0,5**	**1**	**1**	**1**	**7,00**

La nota de la primera fase consiste en el promedio simple de 4 notas {**Razón y justificación, Interlocutores, Necesidades, Requisitos**} donde cada uno de estas notas corresponden a una sección en particular

Nota Entrega 1 = Prom {**Razón y justificación, Interlocutores, Necesidades, Requisitos**}. Esto por cada evaluador. Nota Final Entrega 1 = Prom(NotaEvaluador1 a 3)

Contenido de entrega:

- Deben enviar PPT (en pdf) con lo establecido en esta diapositiva
- Ese día deben presentar un resumen de su entrega (15 min máx)
- Informe (máx 5 páginas) - Template - Comparto hoy

A continuación se presenta en detalle los artefactos de la Entrega 1

Objetivo Entrega 1

1. Objetivo
    - Tener la **Visión APROXIMADA** del producto.
2. ¿Cómo se podría definir un enfoque metodológico de desarrollo de software?
3. ¿Qué objetivos busca resolver con un enfoque metodológico?
4. ¿Qué elementos puede incluir un enfoque metodológico?

Primera Fase
Crear la visión del proyecto

1. El caso de negocio es revisado para crear una declaración sobre la visión del proyecto que servirá de guía y proporcionará el enfoque al proyecto
    - La visión del proyecto debe enfocarse en el problema y no en la solución
    - Debe ser lo suficientemente flexible para adaptarse a futuros cambios conforme avanza el proyecto

Crear la visión del proyecto

Declaración de la visión *Describir la visión general del proyecto de una manera clara y concisa.*			
Grupo Objetivo 	Necesidades 	Producto / Servicio 	Valor 
¿A cuál segmento del mercado será dirigido el producto? *¿Quiénes serán los posibles compradores?*	*¿Qué necesidad estará siendo cubierta por el producto o servicio?* *¿Qué tipo de beneficios generará?*	*Mencionar entre 3 y 5 características principales del producto o servicio que cubren la necesidad identificada y lo hacen único*	*¿Cuáles serán los beneficios para la empresa?* *Ejemplo: Aumento de rentabilidad, reducción de costos, posicionamiento, etc.*

Contenido documento Visión (1)

1. Introducción y Contexto
2. Problema y/o Motivación
3. Valor de Negocio

Razón y Justificación

4. Descripción de los interesados
   a) Resumen de los interesados
   b) Resumen de usuarios

Interlocutores relevantes

Contenido documento Visión (2)

4. Descripción del producto
   a) Resumen de rasgos y beneficios
   b) Suposiciones y referencias
   c) Modelo de Proceso
5. Riesgos
6. Detalle rasgos de la solución
   a) Requisitos funcionales y beneficios
   b) Restricciones
   c) Requisitos no funcionales
   d) Tecnologías a ocupar
   e) Primer versión arquitectura
   f) Primeras maquetas (mockups) de vistas
7. Plan de trabajo
8. Glosario
9. Anexos

Resumen de necesidades y contexto del cliente

Detalle requisitos

Nota Final E1 por cada alumno:

E1 = 90% trabajo evaluado + 10% evaluación pares

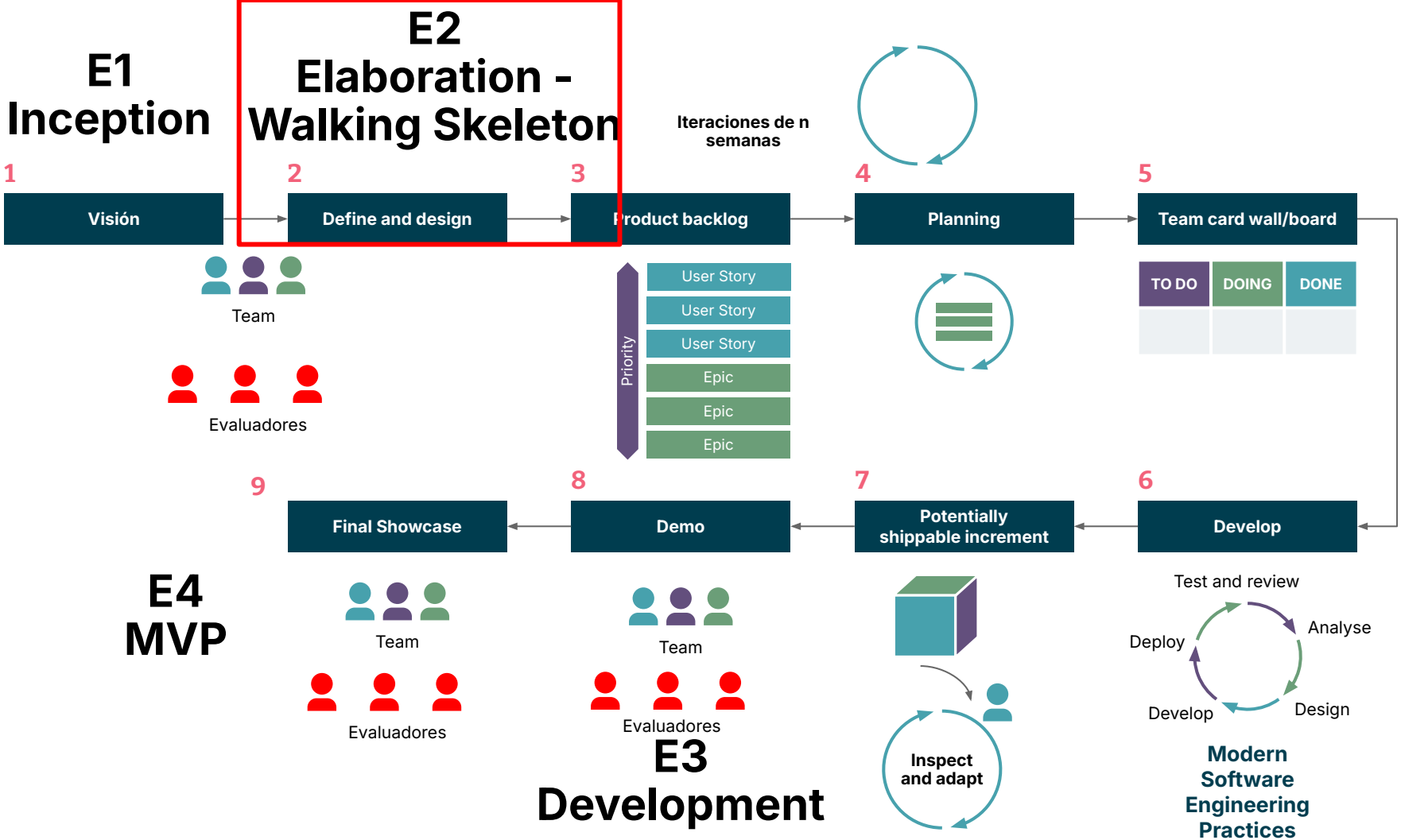
Proyecto 2da Entrega Elaboración

Gonzalo Martínez
gonzalo.martinez@uc.cl



Nota Final E1 por cada alumno:

E1 = 90% trabajo evaluado + 10% evaluación pares


E1
Inception
E2
Elaboration -
Walking Skeleton
Iteraciones de n
semanas
1
Visión
2
Define and design
3
Product backlog
4
Planning
5
Team card wall/board
Team
Evaluadores
Priority
User Story
User Story
User Story
Epic
Epic
Epic
TO DO
DOING
DONE
9
Final Showcase
8
Demo
7
Potentially
shippable increment
6
Develop
E4
MVP
Team
Evaluadores
Team
Evaluadores
E3
Development
Inspect
and adapt
Test and review
Analyse
Design
Develop
Deploy
Modern
Software
Engineering
Practices

Checklist Entrega 2

- Casos de usos y Requerimientos *
- Arquitectura *
- Modelo de dominio
- **Modelo de datos**
- Lista riesgos actualizado *
- Plan de trabajo/desarrollo actualizado *
  - **¿Qué se encuentra completo y que falta?**
  - Tomar en cuenta feedback entrega 1
- Evidencia de CI/CD Walking Skeleton (Pantallazos, Gifs o Live demo)
  - Repositorios ya creados en Github, Gitlab u otro.
  - Implementar un simple “Hello World” de backend y frontend.
  - **Un Continuous Integration (CI) básico de Backend y Frontend.**
  - En lo posible un simple Continuous Deploy (CD) implementando para backend, frontend o ambos (lo mejor es que sea en ambos). Peor caso: manual

*** : aspectos ya evaluados en entrega 1. PERO sí se encontraron observaciones deben ser corregidas y presentadas de nuevo en esta entrega.**

Si no hay observaciones, dejen estos aspectos en un anexo de la entrega 2, por ejemplo, slides de anexo.

Entrega 2 - Walking Skeleton

- Hello world en frontend conectado a backend.
- Junto con un pipeline CI/CD (tag + release en github)
- CI: con stages definidos
- CD: despliegues automatizados hacia producción
    - Si no se puede automatizado, argumentar en la presentación porque no se puede o no se realizará. Sin embargo, si no es automatizado debe ser manual.

Pauta Entrega 2 - Elaboración

- En esta entrega hay aspectos que se repiten, tales como, Riesgos, Arquitectura y Plan de Desarrollo. Esto es debido a que **si en la primera entrega se encontraron defectos en estos apartados ahora deben volver a presentarlos corregidos**.
- En caso de que en la entrega anterior no se encontraron defectos en estos apartados, se mantienen tal cual y lo dejan en un anexo de la presentación -no lo muestran en la presentación-
- **No deben presentar aspectos de la entrega pasada, tales como, introducción, motivación, interlocutores, modelos de proceso, riesgos u otros.**

Evaluador							
Casos de Uso y Requerimientos	Arquitectura	Modelo de dominio	Modelo de datos	Lista Riesgos	Plan de Desarrollo	CI/CD Walking Skeleton	Nota
0,50	0,75	0,75	2,00	0,50	0,50	1,00	7,0

Nota Final E2 por cada alumno:

E2 = 80% trabajo evaluado + 20% evaluación pares

Máximo 10-15 minutos de presentación por equipo + 5-10 minutos de preguntas

No es necesario mostrar su app funcionando
En esta entrega no evaluamos eso

Deben tomar todo el feedback entregado y trabajar en ello

¿Qué se entrega?

Documento:

- PPT (obligatorio) + informe en pdf (opcional)
- Planilla excel con estimaciones y plan de trabajo

¿Qué se entrega?

- Las tablas tipo planilla excel puede incluirlas en su PPT o informe docx. También puede enviarla de forma separada

Parte de esta entrega ya la realizaron en la entrega 1
Ej: arquitectura

Fase de Elaboración

→ Objetivo: Mitigación de Riesgos

- Hito de cierre de la fase (Objetivo), responder las preguntas:
  - ¿La visión del producto está estable?
  - ¿La arquitectura está estable?
  - ¿Demo ejecutable a entregar resuelve los principales riesgos?
  - ¿Próxima fase está suficientemente detallada?
  - ¿Estructura de costos aceptable?

Artefactos Cierre Elaboración

RESUMEN:

1. Actualización Casos de Uso de Relevantes → Especificación
   - Descripción con el nivel de detalle adecuado a cada caso de uso, según complejidad y/o relevancia.
   - Incluir descripción de requisitos no-funcionales como subcapítulo.
2. Documentación de Diseño Preliminar:
   - Modelo de Dominio con los objetos más relevantes y sus relaciones.
3. Plan de Desarrollo.
4. Riesgos Actualizados.

Contenido

- Objetivos e Hitos de Cierre Fase Elaboración
- Documentación:
  1. Actualización de Casos de Uso Relevantes
  2. Modelo de Arquitectura
  3. Modelo de Dominio (y Modelo de Datos)
  4. Plan de Desarrollo
  5. Lista de Riesgos Actualizada

1. Casos de Uso Actualizados

- Deben estar todos los casos de uso relevantes, adecuadamente documentados.
- Aún pueden ser sujeto de cambios durante la construcción, pero no pueden terminar Elaboración sin un detalle adecuado para incorporarlos uno por uno en el plan de desarrollo.
- Pueden agregarse posteriormente algunos casos de uso menos relevantes y posibles flujos alternativos.

1. Casos de Uso Actualizados

- Formato y Contenido:
  - Puede ser un documento separado o un capítulo del documento correspondiente a la entrega.
  - Se comienza con una introducción y explicación general del contexto funcional. Esta explicación puede ser acompañada por un diagrama UML de casos de uso incluyendo los más relevantes.
  - Luego se detallan uno por uno
  - Agregar como sección final una enumeración de requisitos no-funcionales.

1. Casos de Uso Actualizados

CONTENIDO DE UN CASO DE USO:

- Nombre o Título (y número identificación)
- Breve Descripción, incluyendo meta
- Actores involucrados
- Stakeholders
- Precondiciones/postcondiciones
- Triggers
- Flujo de Eventos:
  - Básico (1)
  - Alternativos (N)

Caso de Uso - Ejemplo

- **Inscripción de Usuario (H2.1)**
- **Descripción**:
  Un nuevo usuario se inscribe en el sistema siguiendo la opción desde la página principal, para luego llenar un formulario en el cual se le solicitan varios datos personales, algunos obligatorios como e-mail, nombre completo, dirección postal, teléfono, y otros opcionales como RUT o N° Pasaporte, género, fecha de nacimiento. Se le solicita ingresar una contraseña (dos veces para asegurar que la escribió bien y se valida la correctitud del e-mail.
  La Meta es lograr un registro de inscripción de nuevo usuario y darle una credencial válida para utilizar el sistema.

- **Actores**: Usuario (nuevo)
- **Stakeholders** relevantes del caso:
  - Gerente Operaciones (“temas seguridad y almacenamiento de cuentas”)
  - Gerente Comercial (“facilidades de registro”).
- **Precondiciones**: (ninguna).
- **Triggers**: al terminar correctamente se deja un registro en el *log*.

Caso de Uso - Ejemplo

- Flujo Básico:
  1. Se abre formulario al usuario.
  2. Usuario llena los datos completos.
  3. Usuario realiza registro de inscripción.
  4. Se muestra mensaje de “inscripción exitosa”.

- Flujos Alternativos (debe detallarse cada uno):
  - Caso de falta de uno o más datos obligatorios.
  - Falla en validación formato de e-mail.
  - E-mail de usuario ya se encuentra registrado.
  - Falla conexión a BD.
  - Otros …

2. Doc. de Diseño Preliminar

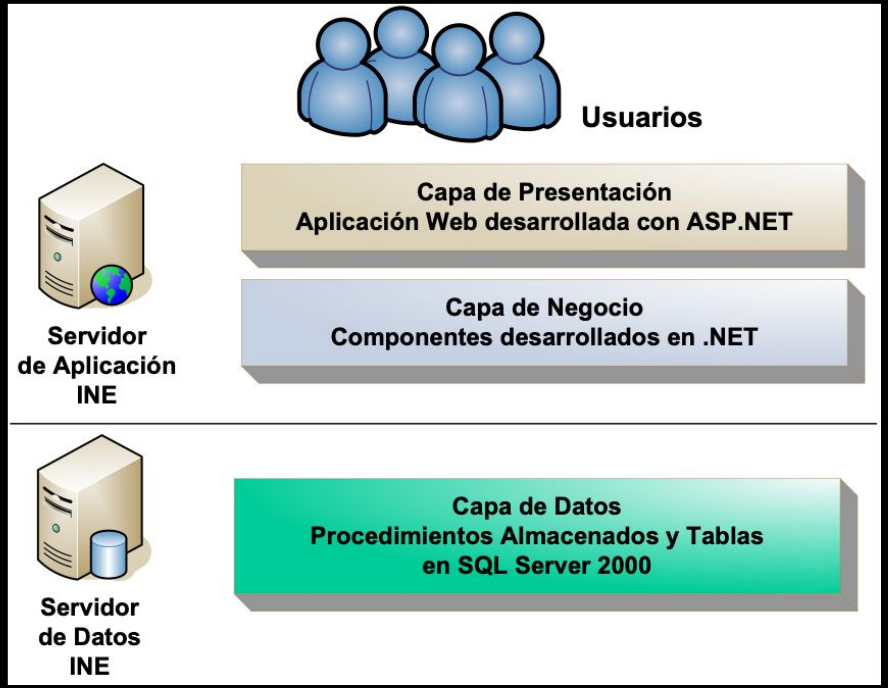
- El objetivo es haber imaginado detalles relevantes de la solución.
- No se espera tener un documento de diseño detallado, pero si tener una idea clara de cómo se implementará la solución.
- El formato es un único documento, con subcapítulos que describen los elementos más fundamentales.

2.1 Modelo de Arquitectura

- Formato:
  - Subcapítulo "Modelo de Arquitectura"
  - Comienza por una explicación o introducción relacionada al contexto del proyecto y restricciones de plataforma posiblemente existentes por parte del cliente.
  - Se incluye un diagrama arquitectural, adecuado al caso del proyecto y se agrega la descripción que corresponde a este caso.
  - Idealmente documentar brevemente la realización de una demo, prueba de concepto, o prototipo con la arquitectura propuesta.

2.1 Modelo Arquitectura - Ejemplo

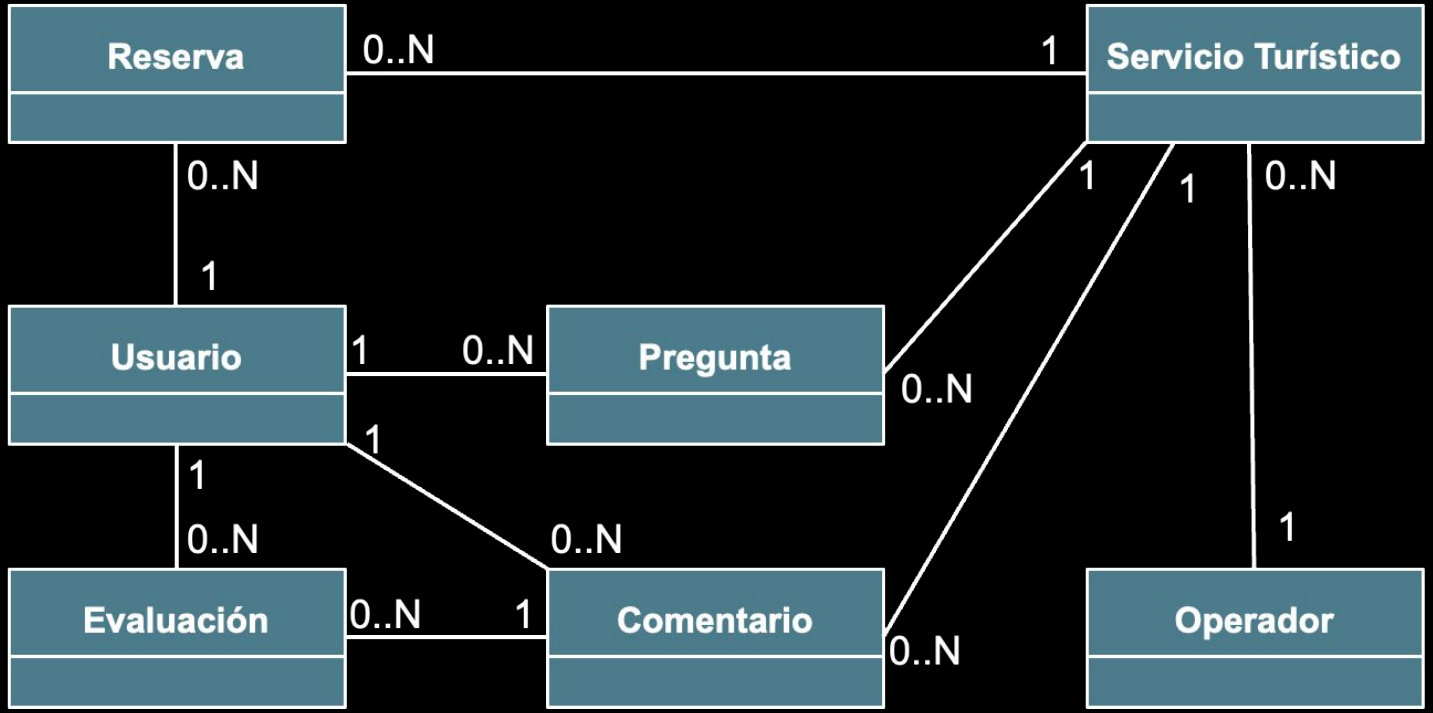
- Ejemplo para arquitectura 3 capas / 2 niveles



2.2 Modelo de Dominio

- Basándose en el formato de un diagrama de clases UML, el modelo de dominio cubre los elementos fundamentales de la solución.
- Constituye una representación visual de **clases conceptuales** u objetos de la vida real en el contexto del proyecto.
- Lo importante de hacer el modelo de dominio, es que como ejercicio realizado por TODOS los integrantes del equipo, ayuda a discutir, visualizar, y refinar la solución.
- Se espera que la versión entregada haya pasado por varias revisiones intermedias con la participación del equipo.
- No necesariamente se traduce en las clases definitivas, pero es el punto de partida, cuando los casos de uso fundamentales ya están estables.

2.2 Modelo de Dominio - Ejemplo



3. Plan de Desarrollo

- El plan debe describir con un detalle adecuado los hitos relevantes, calendario principal, estimaciones gruesas y asignación de tareas por recursos.
- Puede dar espacio a cambios posteriores, en especial en iteraciones finales, pero debe proponer las fechas de cada iteración, que serán inamovibles.
- Además debe describir en forma general cómo se realizará el trabajo dentro de estas iteraciones.

3. Plan de Desarrollo

- Formato:
  - Puede ser C. Gantt, Sprint Plan de Scrum, hitos, u otro. Debe ser claro, preciso, conciso, no-ambiguo.
  - Se debe hacer la priorización de features en base a prioridad y esfuerzo
  - Se sugiere utilizar plantilla Excel de Sprint Plan de Scrum (ejemplo siguiente slide).
  - El plan debe incluir casos de uso y posiblemente otras tareas no asociadas a funcionalidades (por ej, "Tareas de preparación", "Tareas de Instalación")

3. Plan de Desarrollo

- Como referencia podría usar el siguiente ejemplo donde se detallan casos de uso o funcionalidades generales, prioridad, un esfuerzo asociado a cada caso, y un calendario de entrega por iteración.
- Ejemplo:

Caso de Uso / *Actividad*	**Prioridad**	**Esfuerzo**	**Iter 1**	**Iter 2**	**Iter 3**
Caso de Uso Num 1	**Alta**	2	**2**	**0**	**0**
Caso de Uso Num 2	**Alta**	2	**2**	**0**	**0**
Caso de Uso Num 3	**Media**	1	**1**	**1**	**0**
Configuración ambiente testing	**Alta**	2	**2**	**0**	**0**
Instalación en Cliente	**Alta**	2	**2**	**2**	**2**

3. Plan de Desarrollo

- La estimación de esfuerzo de un caso de uso incluye tiempo de codificación, testing, correcciones.

4. Riesgos Actualizados

- Completar y actualizar la misma planilla de riesgos entregada anteriormente.
  - Se debe dejar una breve constancia de los planes de mitigación en ejecución.
  - Se debe indicar aquellos riesgos que ya se materializaron y se les está aplicando el plan de contingencia.
  - Se deben actualizar las probabilidades e impacto si corresponde.
- Formato:
  - Mismo formato tabla visto en clases, como subcapítulo o anexo del documento de entrega de la fase.

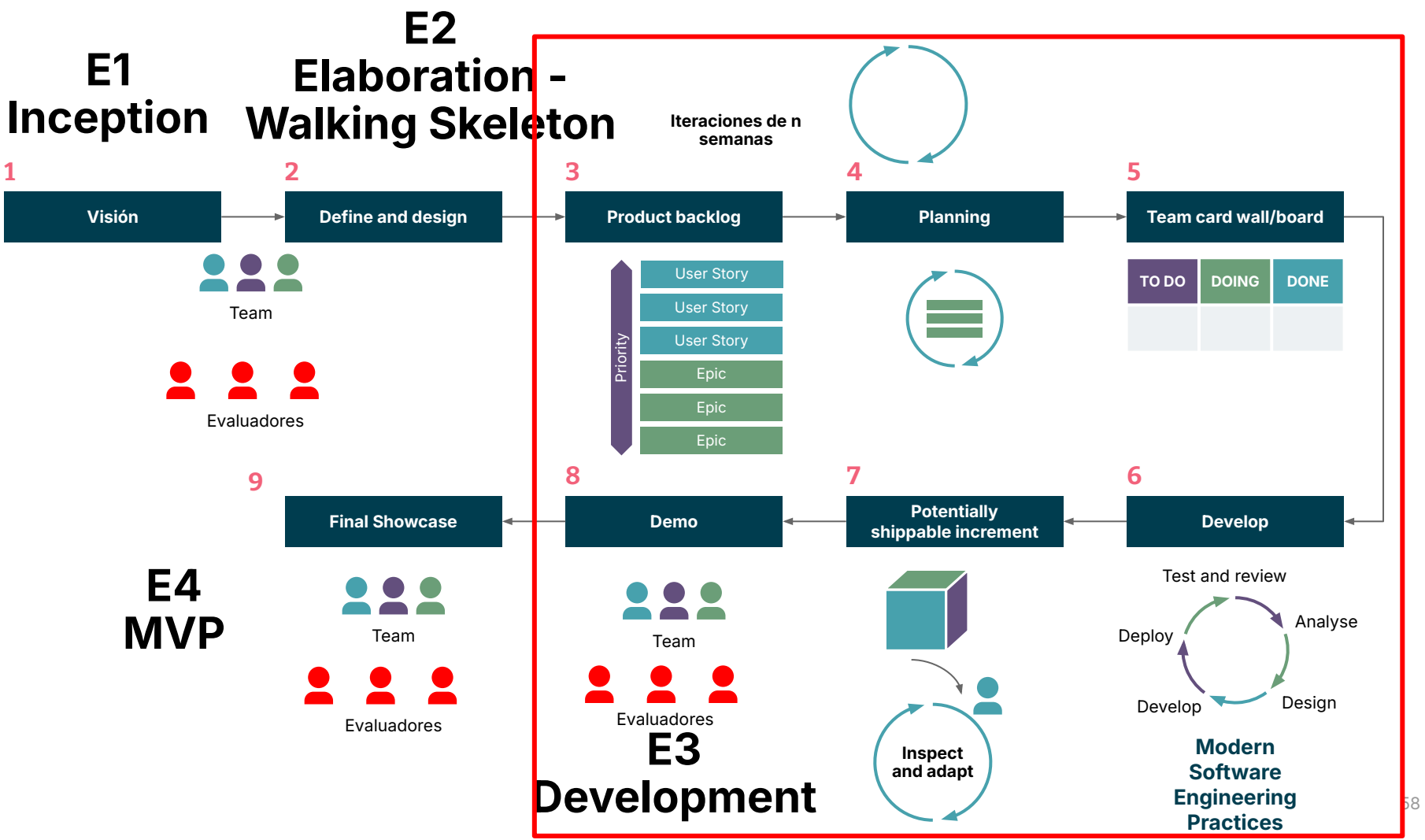
Evaluación de la entrega

- Orden de la entrega.
- Completitud, incluyendo todos los ítems solicitados.
- Correctitud de los elementos.
- Credibilidad y fundamentación del plan y estimaciones globales.
- Atrasos: Descuento 1 punto por cada 2 horas.

Proyecto 3era Entrega Development

Gonzalo Martínez
gonzalo.martinez@uc.cl




E1
Inception
E2
Elaboration -
Walking Skeleton
Iteraciones de n
semanas
1
Visión
2
Define and design
3
Product backlog
4
Planning
5
Team card wall/board
Team
Evaluadores
Priority
User Story
User Story
User Story
Epic
Epic
Epic
TO DO
DOING
DONE
9
Final Showcase
8
Demo
7
Potentially
shippable increment
6
Develop
E4
MVP
Team
Evaluadores
Team
Evaluadores
E3
Development
Inspect
and adapt
Test and review
Analyse
Design
Develop
Deploy
Modern
Software
Engineering
Practices

Checklist Entrega 3

- Evidencias de su proceso de desarrollo
- Uso de métricas y análisis de cómo han llevado a cabo su proceso de desarrollo
  - Deben responder la pregunta: ¿Alcanzan?
- Muestra y evidencia de las prácticas que han utilizado a nivel de desarrollo de software, ingeniería de software y testing
- Muestra y evidencia de CI/CD
- Plan de trabajo actualizado
  - ¿Qué se encuentra completo y que falta?
  - **Deben responder la pregunta: ¿Alcanzan?**
- Mostrar una demo del estado actual de su proyecto (no importa el ambiente)

Pauta Entrega 3 - Development (Aprox 20 minutos de presentación + demo y 10-15 min de preguntas)

Proceso de desarrollo	Uso de Métricas Objetivas	Prácticas y Estado del Código	CI/CD y Tests	Plan de Trabajo y Avances	Demo y Revisión Repositorio	**Nota**
1,00	1,00	1,00	1,00	1,00	1,00	**7,0**

Nota Final E3 por cada alumno:

E3 = 70% trabajo evaluado + 30% evaluación pares

Entrega 3 - En detalle

- Esta entrega se enfoca en **medir cómo se encuentra el estado actual de su desarrollo**.
- Las preguntas a resolver en esta entrega son:
  - ¿Cómo ha sido su ciclo de desarrollo hasta el momento?
  - ¿Cómo han garantizado la calidad de su producto?
  - ¿Qué prácticas han usado? ¿Qué beneficio les ha traído?
  - ¿Hay alguna práctica que dejaron de usar?
  - ¿Qué decisiones han tomado? ¿Por qué?
  - **¿Alcanzan a terminar lo que prometieron?**
  - **¿Si no alcanzan, qué medidas están tomando y qué medidas tomarán?**

¿Cómo ha sido su proceso de desarrollo? (Evidencias del proceso de desarrollo)

- ¿Qué prácticas han utilizado? ¿Qué han logrado?
  - ¿Cómo han manejado la deuda técnica?
  - ¿Cuál es su proceso de resolución de bugs?
  - ¿Cómo han documentado?, ¿ADR?, ¿Diagramas?
    - Mostrar evidencia.
  - ¿Cómo han realizado su estrategia de testing?
  - ¿Qué porcentaje de cobertura de tests tienen hasta el momento? ¿Para ustedes cuánto sería lo óptimo?
  - ¿Cuándo una card está done?
  - ¿Cuando una card avanza de estado?

¿Alcanzan a terminar lo que prometieron?

- Argumentar en base a **medidas objetivas**, tales como:
  - Métricas
  - Proyecciones
  - Gráficos
  - Burnup chart, Burndown chart
  - En general pueden usar cualquier métrica vista en el curso u otra, o cualquier artefacto de alguna metodología que entregue una visión objetiva de lo que está pasando en su proceso de desarrollo

Ejemplo de Métricas

- Métricas de equipo
  - Lead Time
  - Touch Time
  - Efficiency
  - Cards terminadas vs Cards en Backlog
  - No solo fijarse en cards terminadas
    - En 1 semana terminamos 10 cards. ¿Pero qué tipo de cards?
  - Cantidad de Spikes
  - ¿Cuanto tiempo demoran en terminar una card?
  - ¿Cuánto tiempo disponen y han invertido en el proyecto?
  - Pueden ocupar otras o incluso pueden crear sus propias métricas

Estado del Código

- ¿Cómo han mantenido la calidad del proyecto?
  - ¿Sanidad del código, cómo lo mantienen?
  - ¿Cuál es su cantidad de defectos?, ¿Cómo lo han trabajado?
  - ¿El producto en el ambiente de desarrollo se encuentra estable?
- Mostrar evidencia de Pipeline CI/CD (imagen)

Plan de trabajo

- ¿Cómo ha evolucionado su plan de trabajo?
  - ¿Algún cambio? ¿Por qué?
  - Decisiones en base a plan de trabajo
  - Tomar en cuenta feedback entregado
- Mostrar plan de trabajo de cara al MVP
  - Que se encuentra completo
  - Qué falta

¿No alcanzan?
TIPS

- Disminuir tamaño del software.
  - **Priorizar lo más importante a nivel de negocio**
- Incrementar la productividad
  - ¿Cómo?
- Asumir que la fecha original esta pérdida.
  - **Volver a planificar de forma realista**.
- Repensar sus prácticas, ¿cuáles son prudentes de adoptar y cuáles no?
- **Lean Development**
  - **Sacar toda la grasa e ir al grano**
- **Analicen su situación actual**

En resumen, deben entregar

- Evidencias de su proceso de desarrollo (**fotos de Pull Request, stages de CI/CD en el PR, cómo realizan merges de pull request**).
- **Uso de métricas y análisis de cómo han llevado a cabo su proceso de desarrollo**
  - **Deben responder la pregunta: ¿Alcanzan a terminar?**
- **Muestra y evidencia de las prácticas que han utilizado a nivel de desarrollo de software, ingeniería de software y testing**
  - Testing deben responder las siguientes preguntas
    - **¿Cuál ha sido su estrategia de testing? ¿Por qué? ¿Cómo realizan tests en el backend y en el frontend? ¿Cuál es el coverage actual?**
- Muestra y evidencia de CI/CD (**Foto de pipeline + stages, mostrar en vivo**)
- **Plan de trabajo actualizado**
  - **¿Qué se encuentra completo y que falta?**
  - **Deben responder la pregunta: ¿Alcanzan?, caso contrario, ¿Qué van a priorizar?**
- **Mostrar una demo del estado actual de su proyecto (no importa el ambiente)**

Importante:
Esta es la última oportunidad para decir si alcanzan o no
Debe ser justificado objetivamente

Importante:
La siguiente entrega vamos a cobrar todo lo que prometieron

Importante:
Breve demo del software actual funcionando, ya sea local o en prod

¿Qué se entrega?

Documento:

- PPT (obligatorio) + informe pdf (opcional)
- Planilla excel con estimaciones y plan de trabajo

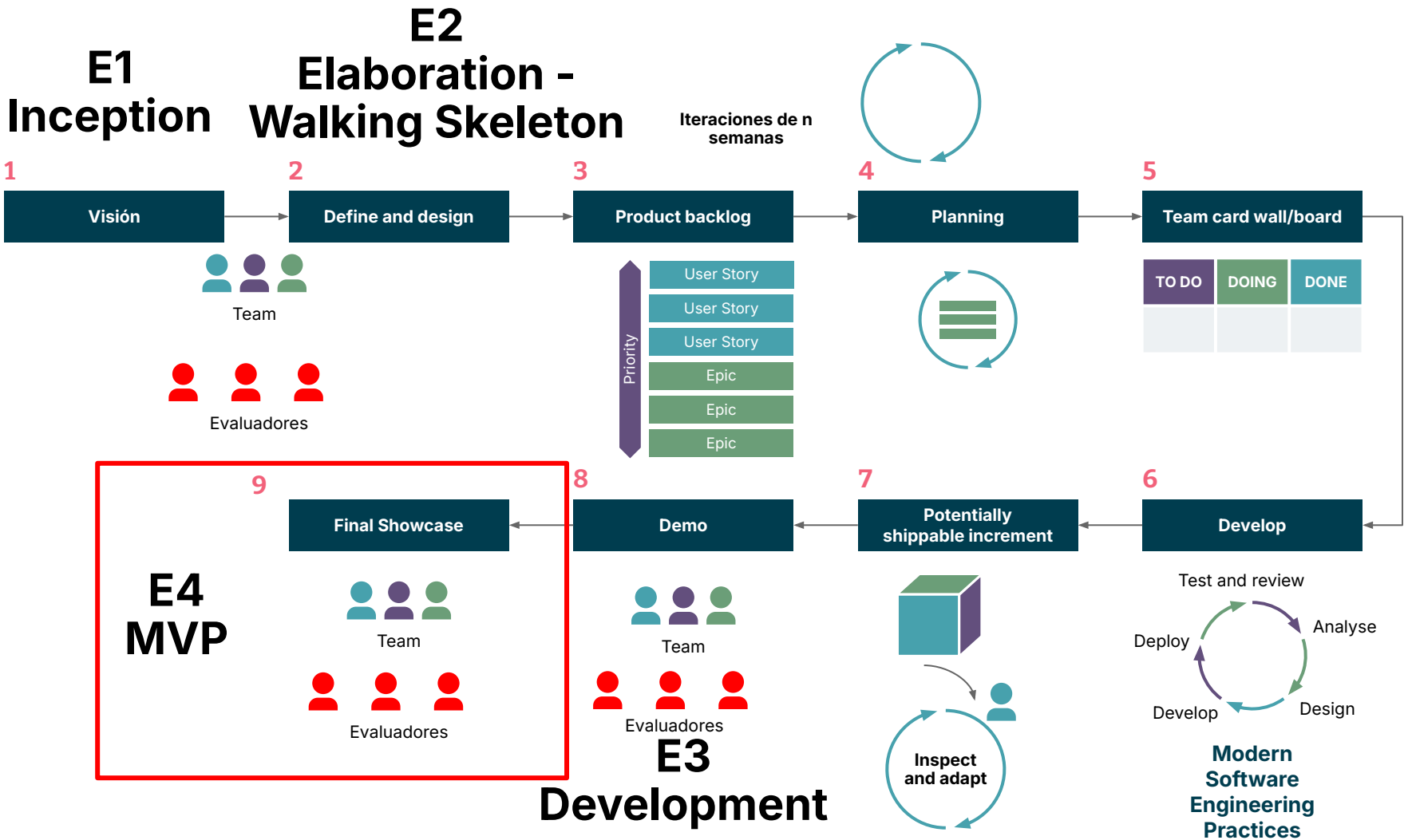
¿Qué se entrega?

- Las tablas tipo planilla excel puede incluirlas en su PPT o informe docx. También puede enviarla de forma separada

Proyecto Última Entrega MVP

Gonzalo Martínez
gonzalo.martinez@uc.cl




E1
Inception
E2
Elaboration -
Walking Skeleton
Iteraciones de n
semanas
1
Visión
2
Define and design
3
Product backlog
4
Planning
5
Team card wall/board
Team
Evaluadores
Priority
User Story
User Story
User Story
Epic
Epic
Epic
TO DO
DOING
DONE
9
Final Showcase
8
Demo
7
Potentially
shippable increment
6
Develop
E4
MVP
Team
Evaluadores
Team
Evaluadores
E3
Development
Inspect
and adapt
Test and review
Analyse
Design
Develop
Deploy
Modern
Software
Engineering
Practices

Checklist MVP

- Esta entrega se enfoca en **entregar y dar por finalizado el MVP**.
- Los entregables son:
  - **1. Foto/Diagrama de la Arquitectura final + Modelo de datos (1-2 slides max) En anexo diagramas de proceso corregidos**
  - **2. Prácticas y estado del código (tests)**
    - ¿Cómo aseguran la calidad de este MVP? **(1-2 slides max)**
  - **3. Plan de trabajo actualizado y finalizado**
    - A partir del último plan resumido de trabajo, **mostrar que se terminó y que quedó pendiente (1-2 slides max)**
    - Deben responder:
      - ¿Alcanzaron? ¿Qué decisiones tomaron?
  - **4. Análisis de lo que fue todo su proceso y trabajo (1-2 slides max, ver siguiente slide para más detalle)**
  - **5. Live Demo de todas las funcionalidades/casos de uso comprometidos**
    - **Demo en vivo**

Checklist MVP

- Esta entrega se enfoca en **entregar y dar por finalizado el MVP**.
- Los entregables son:
  - **Entregar todo el material realizado en el curso con las correciones/feedback entregado a lo largo del semestre.**

4. Análisis de su proceso (1-2 slide max)

- En este punto nos interesa saber cuál es su conclusión sobre todo lo que fue el desarrollo de este proyecto a lo largo del curso
- Queremos saber su visión objetiva acerca de:
  - **1. ¿Qué hicieron bien?**
  - **2. ¿Qué no volverían a repetir?**
  - **3. ¿Errores y aciertos?**
    - Por ejemplo; íbamos lentos pero dejamos de hacer X o aplicamos Y lo cual trajo la ventaja Z
    - Por ejemplo; no debíamos dar tanto esfuerzo a la feature X porque no era tan importante a nivel de negocio.
  - No queremos respuestas simples del tipo: “aprendimos que el trabajo en equipo es importante”, si no, un análisis más profundo a nivel de ingeniería sobre las decisiones tomadas.
  - **Para resolver estas preguntas, les recomiendo hacer una reunión de retrospectiva final.**

Pauta Evaluación Final
(30 minutos aprox de presentación)
Nota Final = Punto Base (1) + (suma de Ponderación * PuntajeApartado)

Arquitectura	Prácticas, Cálidad y Estado del Código	Plan de Trabajo	Conclusión: Su propio análisis de lo que fue todo el proceso de desarrollo. ¿Qué hicieron bien? ¿Qué no volverian a repetir? ¿Errores y Aciertos? ¿Aprendizajes?	Demo y Revisión Repositorio	NOTA PRESENTACIÓN FINAL
0,25	0,50	0,25	2,00	3,00	
1,00	1,00	1,00	1,00	1,00	*7,00*

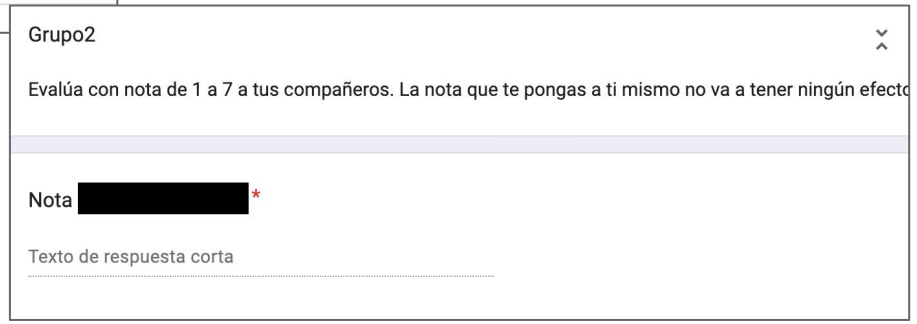
NOTA PRESENTACIÓN (40%)	NOTA PROMEDIO EVALUACIÓN PARES (60%)	NOTA FINAL ENTREGA 4
7	7	7

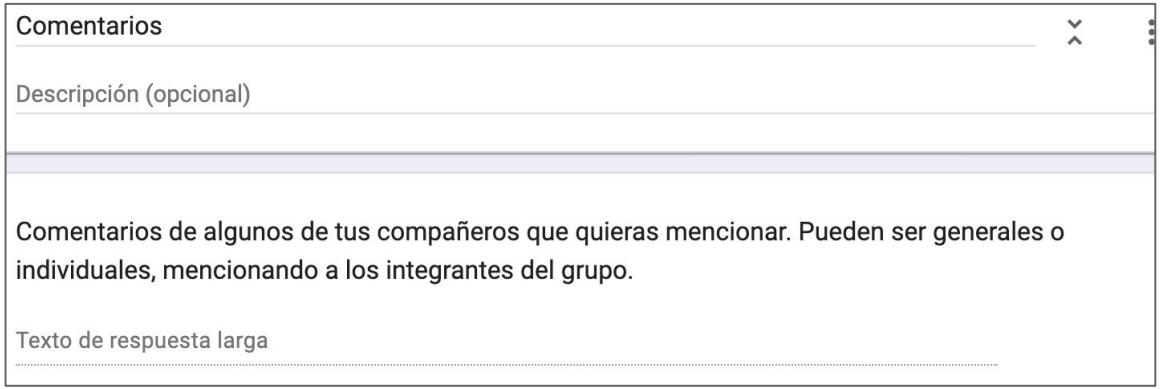
Nota Final E4 por cada alumno:

E4 = 50% trabajo evaluado + 50% evaluación pares

Evaluación Pares (Encuesta Google Form)







¿Qué se entrega?

- Presentación (PPT o PDF) con todas las slides previamente mencionadas
- Enviar PPT/PDF el mismo día de su presentación (puede ser al momento de presentar)

IIC 3143 Desarrollo de Software

Gonzalo Martínez
gonzalo.martinez@uc.cl

